

The Complete Queue & BFS Roadmap

(Beginner → Advanced, Java)

71 curated problems, prioritizing **LeetCode (LC)**, supplemented with **GeeksforGeeks (GfG)** for classic patterns (monotonic deque intro, plain graph BFS, topological sort) that LeetCode doesn't isolate as cleanly. As with the earlier roadmaps in this series: only LeetCode assigns numeric IDs — GfG problems show "—" in the Problem # column.

Tags: 🔥 Must-Do · ★ Recommended · 💡 Optional · 🏆 Hard Challenge · 📖 Premium

Level 0 — Queue Fundamentals & Implementation

Patterns: Queue Fundamentals, Queue Implementation, Circular Queue

Before problems: **implement a queue yourself** backed by an array with wraparound indices (this is a circular queue) and backed by a linked list. Understanding why a naive array queue "leaks" capacity without wraparound is the actual prerequisite here.

Order	Platform	Problem #	Problem Name	Difficulty	Pattern	Prerequisite	Tag
1	LeetCode	232	Implement Queue using Stacks	Easy	Queue Implementation	Build-your-own queue	🔥
2	LeetCode	225	Implement Stack using Queues	Easy	Queue Implementation (mechanics)	#1	★
3	LeetCode	622	Design Circular Queue	Medium	Circular Queue	#1	🔥
4	LeetCode	933	Number of Recent Calls	Easy	Queue + Arrays (sliding count)	#1	★

Level 1 — Basic Easy Problems (Queue + Arrays/Strings, Simple Design)

Patterns: Queue + Arrays, Queue + Strings

Order	Platform	Problem #	Problem Name	Difficulty	Pattern	Prerequisite	Tag
5 	LeetCode	346	Moving Average from Data Stream	Easy	Queue + Arrays (sliding sum)	Level 0	
6	LeetCode	1670	Design Front Middle Back Queue	Medium	Queue Design	Level 0	
7 	LeetCode	362	Design Hit Counter	Medium	Queue + Arrays (time-windowed count)	Level 0	
8 	LeetCode	359	Logger Rate Limiter	Easy	Queue + HashMap	Level 0	

Level 2 — Queue + Deque

Pattern: Deque

Order	Platform	Problem #	Problem Name	Difficulty	Pattern	Prerequisite	Tag
9	LeetCode	641	Design Circular Deque	Medium	Deque	Level 0	
10	GfG	—	First negative integer in every window of size k	Medium	Monotonic Deque (intro)	#9	
11	LeetCode	649	Dota2 Senate	Medium	Queue Simulation (two-queue elimination)	Level 0	
12	LeetCode	950	Reveal Cards In Increasing Order	Medium	Queue Simulation	Level 0	

Must-Do & key pattern: #9 forces you to handle both ends with wraparound indices — the actual mechanical skill a `Deque` interface hides from you in Java. #10 is your first real monotonic structure: maintain indices in the deque such that their values stay in a useful (here: increasing) order, popping from the back when a new element invalidates older ones — this exact loop reappears, barely modified, in Level 7.

Level 3 — BFS on Trees

Patterns: *BFS Fundamentals, Binary Tree Level Order Traversal, BFS on Trees*

Order	Platform	Problem #	Problem Name	Difficulty	Pattern	Prerequisite	Tag
13	LeetCode	102	Binary Tree Level Order Traversal	Medium	BFS Fundamentals / Level Order	Level 0	🔥
14	LeetCode	111	Minimum Depth of Binary Tree	Easy	BFS (shortest path on a tree)	#13	🔥
15	LeetCode	107	Binary Tree Level Order Traversal II	Medium	BFS + Reverse	#13	★
16	LeetCode	103	Binary Tree Zigzag Level Order Traversal	Medium	BFS + Direction Flip	#13	🔥
17	LeetCode	637	Average of Levels in Binary Tree	Easy	BFS + Aggregation	#13	★
18	LeetCode	199	Binary Tree Right Side View	Medium	BFS (last-per-level)	#13	🔥
19	LeetCode	515	Find Largest Value in Each Tree Row	Medium	BFS + Aggregation	#13	★
20	LeetCode	429	N-ary Tree Level Order Traversal	Medium	BFS / N-ary	#13	💡
21	LeetCode	116	Populating Next Right Pointers in Each Node	Medium	BFS (level-linking, O(1) space variant exists)	#13	★

Must-Do & key pattern: #13 is the archetype for the entire roadmap — `queue.offer(root)`, then loop `size = queue.size()` times per iteration so you know exactly where one level ends and the next begins. #14 is the first hint that BFS naturally finds *shortest* distances (fewest edges) before DFS would, because BFS explores in strict distance order — this single fact is the seed of everything in Levels 5, 6, and 9.

Level 4 — BFS on Graphs

Pattern: *BFS on Graphs*

Order	Platform	Problem #	Problem Name	Difficulty	Pattern	Prerequisite	Tag
22	GfG	—	BFS of Graph	Easy	Pure Graph BFS Traversal	Level 3	🔥
23	LeetCode	133	Clone Graph	Medium	BFS on Graphs + HashMap	#22	🔥
24	LeetCode	200	Number of Islands	Medium	BFS/DFS on an implicit grid graph	#22	🔥
25	LeetCode	733	Flood Fill	Easy	BFS/DFS basic	#22	★
26	LeetCode	130	Surrounded Regions	Medium	BFS from the boundary inward	#24	★

Must-Do & key pattern: #22/#23 move BFS off of trees (where there's exactly one path to any node) onto general graphs, where a `visited` set becomes mandatory — without it, cycles cause infinite loops. #24 is the moment a 2D grid "clicks" as just another graph where each cell has up to 4 implicit neighbors.

Level 5 — Matrix / Grid BFS

Pattern: *Matrix BFS*

Order	Platform	Problem #	Problem Name	Difficulty	Pattern	Prerequisite	Tag
27	LeetCode	1091	Shortest Path in Binary Matrix	Medium	Matrix BFS (shortest path, 8-directional)	Level 4	🔥
28	LeetCode	1926	Nearest Exit from Entrance in Maze	Medium	Matrix BFS (shortest path, boundary exit)	#27	🔥
29	LeetCode	934	Shortest Bridge	Medium	Matrix BFS (DFS to mark island, then BFS to expand)	#27, Level 4	🔥

Must-Do & key pattern: #27/#28 are single-source shortest-path BFS on a grid — the same `size = queue.size()` per-level loop from Level 3, just with grid neighbors instead of tree children, and a `visited[][]` (or in-place marking) instead of a tree's implicit "no revisits" guarantee. #29 is a genuine two-technique combo: DFS to find and mark one full island, then BFS *outward from that entire island at once* — which is the direct bridge into Level 6.

Level 6 — Multi-Source BFS & Rotting/Spreading Problems

Patterns: Multi-Source BFS, Rotting/Spreading Problems

Order	Platform	Problem #	Problem Name	Difficulty	Pattern	Prerequisite	Tag
30	LeetCode	994	Rotting Oranges	Medium	Multi-Source BFS 🔥 ARCHETYPE	Level 5	🔥
31	GfG	—	Rotten Oranges	Medium	rep of #30	#30	💡
32	LeetCode	542	01 Matrix	Medium	Multi-Source BFS	#30	🔥
33	LeetCode	1162	As Far from Land as Possible	Medium	Multi-Source BFS	#30	🔥
34	LeetCode	1765	Map of Highest Peak	Medium	Multi-Source BFS	#30	★
35	LeetCode	417	Pacific Atlantic Water Flow	Medium	Multi-Source BFS (from two separate boundary sets)	#30	★
36 🔒	LeetCode	286	Walls and Gates	Medium	Multi-Source BFS (the original archetype problem)	#30	★

Must-Do & key pattern: #30 is *the* pattern to master here: instead of starting BFS from one node, **seed the queue with every "starting" cell at once** (every rotten orange), then run standard level-by-level BFS — the level number that finishes the traversal directly equals the answer (minutes elapsed). Every problem in this level is a reskin: seed the queue with all zeros (#32), all land cells (#33), all water cells (#34), or all boundary cells (#35/#36).

Level 7 — Queue + Sliding Window / Monotonic Deque

Patterns: Queue + Sliding Window, Monotonic Queue/Deque, Sliding Window Maximum

Order	Platform	Problem #	Problem Name	Difficulty	Pattern	Prerequisite	Tag
37	LeetCode	239	Sliding Window Maximum	Hard	Monotonic Deque 🔥 CORE	Level 2	🔥
38	LeetCode	1438	Longest Continuous Subarray With Absolute Diff $\leq$ Limit	Medium	Monotonic Deque (two deques: min + max)	#37	🔥
39	LeetCode	1696	Jump Game VI	Medium	Monotonic Deque + DP	#37	🔥
40	LeetCode	862	Shortest Subarray with Sum at Least K	Hard	Monotonic Deque + Prefix Sum	#37	★
41	LeetCode	1425	Constrained Subsequence Sum	Hard	Monotonic Deque + DP	#39	🏆
42	LeetCode	1499	Max Value of Equation	Hard	Monotonic Deque (on a transformed value)	#37	🏆
43	LeetCode	2398	Maximum Number of Robots Within Budget	Hard	Monotonic Deque + Sliding Window	#37	🏆

Must-Do & key pattern: #37 is the second archetype of the whole roadmap (after #30). Maintain a deque of *indices* whose values are decreasing front-to-back; pop from the back whenever a new element is bigger (those old candidates can never be the max again while the new one is in the window), and pop from the front when the front index falls outside the window. Every other problem here changes *what* the deque tracks (a range, a DP value, a transformed equation term) but reuses this exact two-ended eviction logic.

Level 8 — Topological Sort / Kahn's Algorithm

Patterns: Topological Sort using Queue, Kahn's Algorithm

Order	Platform	Problem #	Problem Name	Difficulty	Pattern	Prerequisite	Tag
44	GfG	—	Topological sort	Medium	Topological Sort (intro, DAG assumed)	Level 4	💡
45	LeetCode	207	Course Schedule	Medium	Kahn's Algorithm 🔥 CORE (cycle detection)	Level 4	🔥
46	LeetCode	210	Course Schedule II	Medium	Kahn's Algorithm (produce the actual order)	#45	🔥
47	LeetCode	310	Minimum Height Trees	Medium	Kahn's-style layer trimming (peel leaves inward)	#45	🔥
48	LeetCode	802	Find Eventual Safe States	Medium	Topological Sort (reverse-graph Kahn's)	#45	★
49	LeetCode	1462	Course Schedule IV	Medium	Topological Sort + Reachability	#46	★
50 🔒	LeetCode	269	Alien Dictionary	Hard	Topological Sort (build graph from ordering constraints)	#46	🔥
51	LeetCode	2050	Parallel Courses III	Hard	Topological Sort + DP (longest path in a DAG)	#46	🏆

Must-Do & key pattern: #45's **Kahn's Algorithm** is the queue-based way to topologically sort (or detect a cycle in) a directed graph: compute in-degree for every node, seed the queue with all in-degree-0 nodes, and each time you process a node, decrement its neighbors' in-degrees, enqueueing any that hit zero. If you process fewer nodes than the graph has, a cycle exists. #50 (Alien Dictionary) is the hardest *construction* step in this level — you must first derive the edges themselves from adjacent word comparisons before Kahn's Algorithm even starts.

Level 9 — Shortest Path / 0-1 BFS / State-Space BFS

Patterns: Shortest Path using BFS, 0-1 BFS, State-Space BFS

Order	Platform	Problem #	Problem Name	Difficulty	Pattern	Prerequisite	Tag
52	LeetCode	127	Word Ladder	Hard	State-Space BFS 🔥 CORE (each word is a graph node)	Level 4, Level 5	🔥
53	LeetCode	752	Open the Lock	Medium	State-Space BFS (each lock combination is a node)	#52	🔥
54	LeetCode	433	Minimum Genetic Mutation	Medium	State-Space BFS (rep of #52's idea)	#52	★
55	LeetCode	1197	Minimum Knight Moves	Medium	State-Space BFS (infinite/unbounded grid)	Level 5	★
56	LeetCode	909	Snakes and Ladders	Medium	BFS on an implicit graph (board squares as nodes)	Level 5	★
57	LeetCode	1368	Minimum Cost to Make at Least One Valid Path in a Grid	Hard	0-1 BFS 🔥 CORE	Level 5, Level 7	🔥
58	LeetCode	2290	Minimum Obstacle Removal to Reach Corner	Hard	0-1 BFS	#57	🔥
59	LeetCode	1293	Shortest Path in a Grid with Obstacles Elimination	Hard	State-Space BFS (position + remaining-budget as the state)	Level 5	★

Must-Do & key pattern: #52 is the moment BFS stops looking like "grid/tree traversal" and becomes purely abstract: nodes are *states* (words), edges are *valid transitions* (one letter changed to a real word), and BFS still guarantees the shortest transformation sequence. #57/#58 introduce **0-1 BFS**: when edge weights are only 0 or 1, a plain Deque (push 0-weight edges to the *front*, 1-weight edges to the *back*) gives Dijkstra-quality shortest paths in $O(V+E)$, without needing a full priority queue.

Level 10 — Advanced / Hard Problems (+ Priority Queue Contrast & Scheduling)

Patterns: Priority Queue/Heap vs Queue, Scheduling/Task Processing, Advanced/Hard Queue Problems

Order	Platform	Problem #	Problem Name	Difficulty	Pattern	Prerequisite	Tag
60	LeetCode	621	Task Scheduler	Medium	Queue/Heap Scheduling	Level 0, Heap basics	🔥
61	LeetCode	1834	Single-Threaded CPU	Medium	Priority Queue Scheduling	#60	★
62	LeetCode	973	K Closest Points to Origin	Medium	Priority Queue (contrast: value-order, not arrival-order)	Heap basics	★
63	LeetCode	1129	Shortest Path with Alternating Colors	Medium	Multi-State BFS	Level 9	★
64	LeetCode	815	Bus Routes	Hard	BFS on a derived graph-of-routes	Level 4, Level 9	🏆
65	LeetCode	126	Word Ladder II	Hard	BFS + Backtracking (reconstruct all shortest paths)	#52 (Level 9)	🏆
66	LeetCode	1263	Minimum Moves to Move a Box to Their Target Location	Hard	State-Space BFS (player position + box position as state)	Level 9	🏆
67 🔒	LeetCode	1730	Shortest Path to Get Food	Medium	Matrix BFS capstone (rep)	Level 5–6	💡

Must-Do & key pattern: #60 is the natural bridge from "Queue" to "Priority Queue" — cool-down scheduling needs *counts* processed in a specific order, which a plain FIFO queue can express directly (a cooldown queue of `(task, availableAgainAtTime)` pairs) without needing a heap at all; contrast this with #61/#62 where a Priority Queue (ordering by *value*, not arrival time) is genuinely required. #64 (Bus Routes) is a great "aha" problem: naively BFS-ing stop-by-stop is too slow, so you instead build a graph of *routes* (two routes are "connected" if they share a stop) and BFS on that instead — a good final lesson in recognizing when to transform the graph before running BFS on it.

Queue & BFS Pattern Cheat Sheet

Queue vs Stack

- Queue = FIFO (first in, first out) — the oldest enqueued item is processed first. Use for BFS, level-order processing, task scheduling in arrival order.
- Stack = LIFO (last in, first out) — the most recently pushed item is processed first. Use for DFS, matching/nesting, undo history.
- If order-of-discovery should match order-of-processing (breadth-first, "closest things first"), it's a Queue. If you want to fully resolve the most recent thing before going back to older ones (depth-first), it's a Stack.

Queue vs Deque

- A Deque (double-ended queue) allows $O(1)$ insertion/removal at **both** ends; a plain Queue only allows insertion at the back and removal at the front.
- Use a Deque when you need to evict from *either* end based on a condition — the defining example is a **Monotonic Deque** (Level 7), where you remove stale/dominated elements from the back on insert, and stale out-of-window elements from the front as the window slides.

Queue vs Priority Queue

- A Queue processes strictly in arrival order. A Priority Queue (heap-backed) processes strictly in **value** order, regardless of arrival order.
- Recognize: "process the smallest/largest/closest next" (regardless of when it arrived) → Priority Queue. "Process things in the order they showed up, level by level" → plain Queue.

BFS vs DFS

- BFS explores level-by-level outward from a source — it's the tool of choice whenever you need the **shortest path / fewest steps / minimum number of levels**, because it guarantees discovering closer nodes before farther ones.
- DFS explores as deep as possible before backtracking — better for exhaustive search (all paths, connectivity, cycle structure) where "shortest" doesn't matter, and it uses less overhead for problems that don't need level information.
- Rule of thumb: if the question has the word "shortest," "minimum steps," "fewest operations," or "nearest," reach for BFS first.

Multi-Source BFS

- Recognize: multiple valid "starting points" that should all begin spreading simultaneously (all rotten oranges, all gates, all land cells).
- Core idea: seed the queue with **every** source node before the first BFS level even begins, instead of starting from one node and looping over sources separately (which would be much slower and give wrong "closest source" answers).

Matrix BFS

- Recognize: grid/matrix, "shortest path," "spread," "nearest," 4-directional (or 8-directional) neighbor exploration.

- Core idea: treat each cell as a graph node with up to 4 (or 8) implicit edges to its neighbors; use a `visited[][]` array or mutate the grid in place to avoid revisiting.

Shortest Path BFS

- Works directly (no weights needed) when **all edges have equal weight (or weight 1)** — the level number at which you reach a node is its shortest distance.
- Breaks down when edges have varying positive weights (use Dijkstra) or when weights can be negative (use Bellman-Ford) — see 0-1 BFS below for the special "only two possible weights" case.

Monotonic Queue (Deque)

- Recognize: "maximum/minimum in every sliding window," or any problem where you need the *best remaining candidate* efficiently as a window slides.
- Core idea: keep the deque's values monotonic (increasing or decreasing front-to-back); an incoming element evicts any *worse and now-irrelevant* candidates from the back before being added; expired (out-of-window) indices are evicted from the front.

Topological Sort

- Recognize: "ordering with dependencies," "prerequisite," "must come before," on a **Directed Acyclic Graph (DAG)**.
- Two standard approaches: DFS-based (postorder, then reverse) or **Kahn's Algorithm** (queue-based, see below) — Kahn's is usually preferred in interviews because it naturally detects cycles as a side effect.

Kahn's Algorithm

- Core idea: compute in-degree for every node; seed a queue with all in-degree-0 nodes; repeatedly dequeue a node, "remove" its outgoing edges (decrement neighbors' in-degrees), and enqueue any neighbor that drops to in-degree 0. If the total number of processed nodes is less than the total node count, a cycle exists (no valid topological order).
- Time: $O(V + E)$. Space: $O(V)$.

0-1 BFS

- Recognize: shortest path where every edge weight is **either 0 or 1** (e.g., "cost 0 to move this way, cost 1 to move that way").
- Core idea: use a plain `Deque` instead of a priority queue — push 0-weight-edge destinations to the **front** (they should be processed before farther things already in the deque) and 1-weight-edge destinations to the **back**. This preserves BFS's "process in increasing distance order" property in $O(V+E)$, without the $O(\log V)$ overhead of a heap-based Dijkstra.

Java Templates for Major Patterns

Basic BFS (Level Order)

```

public List<List<Integer>> levelOrder(TreeNode root) {
    List<List<Integer>> res = new ArrayList<>();
    if (root == null) return res;
    Queue<TreeNode> queue = new LinkedList<>();
    queue.offer(root);
    while (!queue.isEmpty()) {
        int size = queue.size();
        List<Integer> level = new ArrayList<>();
        for (int i = 0; i < size; i++) {
            TreeNode n = queue.poll();
            level.add(n.val);
            if (n.left != null) queue.offer(n.left);
            if (n.right != null) queue.offer(n.right);
        }
        res.add(level);
    }
    return res;
}

```

BFS on a Graph

```

public void bfs(int start, Map<Integer, List<Integer>> adj) {
    Set<Integer> visited = new HashSet<>();
    Queue<Integer> queue = new LinkedList<>();
    queue.offer(start);
    visited.add(start);
    while (!queue.isEmpty()) {
        int node = queue.poll();
        // process node
        for (int neighbor : adj.getOrDefault(node, List.of())) {
            if (!visited.contains(neighbor)) {
                visited.add(neighbor);
                queue.offer(neighbor);
            }
        }
    }
}

```

Multi-Source BFS

```

public int rottingOranges(int[][] grid) {
    int rows = grid.length, cols = grid[0].length, fresh = 0, minutes = 0;
    Queue<int[]> queue = new LinkedList<>();
    for (int r = 0; r < rows; r++)
        for (int c = 0; c < cols; c++) {
            if (grid[r][c] == 2) queue.offer(new int[]{r, c});
            else if (grid[r][c] == 1) fresh++;
        }
    int[][] dirs = {{0,1},{0,-1},{1,0},{-1,0}};
    while (!queue.isEmpty() && fresh > 0) {
        int size = queue.size();
        for (int i = 0; i < size; i++) {
            int[] cell = queue.poll();
            for (int[] d : dirs) {
                int nr = cell[0] + d[0], nc = cell[1] + d[1];
                if (nr >= 0 && nr < rows && nc >= 0 && nc < cols && grid[nr][nc] ==
1) {
                    grid[nr][nc] = 2;
                    fresh--;
                    queue.offer(new int[]{nr, nc});
                }
            }
        }
        minutes++;
    }
    return fresh == 0 ? minutes : -1;
}

```

Monotonic Deque (Sliding Window Maximum)

```

public int[] maxSlidingWindow(int[] nums, int k) {
    Deque<Integer> deque = new ArrayDeque<>(); // stores indices, values decreasing
    int[] res = new int[nums.length - k + 1];
    for (int i = 0; i < nums.length; i++) {
        while (!deque.isEmpty() && nums[deque.peekLast()] < nums[i]) deque.pollLast();
        deque.offerLast(i);
        if (deque.peekFirst() <= i - k) deque.pollFirst();
        if (i >= k - 1) res[i - k + 1] = nums[deque.peekFirst()];
    }
    return res;
}

```

Kahn's Algorithm (Topological Sort)

```

public int[] topoSort(int n, int[][] edges) {
    List<List<Integer>> adj = new ArrayList<>();
    int[] indegree = new int[n];
    for (int i = 0; i < n; i++) adj.add(new ArrayList<>());
    for (int[] e : edges) { adj.get(e[0]).add(e[1]); indegree[e[1]]++; }

    Queue<Integer> queue = new LinkedList<>();
    for (int i = 0; i < n; i++) if (indegree[i] == 0) queue.offer(i);

    int[] order = new int[n];
    int idx = 0;
    while (!queue.isEmpty()) {
        int node = queue.poll();
        order[idx++] = node;
        for (int nxt : adj.get(node)) {
            if (--indegree[nxt] == 0) queue.offer(nxt);
        }
    }
    return idx == n ? order : new int[0]; // empty array signals a cycle
}

```

O-1 BFS

```

public int zeroOneBFS(int[][] grid) {
    int rows = grid.length, cols = grid[0].length;
    int[][] dist = new int[rows][cols];
    for (int[] row : dist) Arrays.fill(row, Integer.MAX_VALUE);
    Deque<int[]> deque = new ArrayDeque<>();
    deque.offerFirst(new int[]{0, 0});
    dist[0][0] = 0;
    int[][] dirs = {{0,1},{0,-1},{1,0},{-1,0}};
    while (!deque.isEmpty()) {
        int[] cell = deque.pollFirst();
        int r = cell[0], c = cell[1];
        for (int[] d : dirs) {
            int nr = r + d[0], nc = c + d[1];
            if (nr < 0 || nr >= rows || nc < 0 || nc >= cols) continue;
            int weight = (grid[nr][nc] != grid[r][c]) ? 1 : 0; // problem-specific w
            eight rule
            if (dist[r][c] + weight < dist[nr][nc]) {
                dist[nr][nc] = dist[r][c] + weight;
                if (weight == 0) deque.offerFirst(new int[]{nr, nc});
                else deque.offerLast(new int[]{nr, nc});
            }
        }
    }
    return dist[rows - 1][cols - 1];
}

```

Top 20 Must-Do Queue Problems

1. LC 232 — Implement Queue using Stacks
2. LC 622 — Design Circular Queue
3. LC 102 — Binary Tree Level Order Traversal
4. LC 111 — Minimum Depth of Binary Tree
5. LC 200 — Number of Islands
6. LC 994 — Rotting Oranges
7. LC 542 — 01 Matrix
8. LC 1091 — Shortest Path in Binary Matrix
9. LC 239 — Sliding Window Maximum
10. LC 1438 — Longest Continuous Subarray With Absolute Diff $\leq$ Limit
11. LC 207 — Course Schedule

12. LC 210 — Course Schedule II
13. LC 127 — Word Ladder
14. LC 752 — Open the Lock
15. LC 1368 — Minimum Cost to Make at Least One Valid Path in a Grid
16. LC 2290 — Minimum Obstacle Removal to Reach Corner
17. LC 269 — Alien Dictionary 🔒
18. LC 621 — Task Scheduler
19. LC 133 — Clone Graph
20. LC 199 — Binary Tree Right Side View

Top 30 Interview Problems

(The 20 above, plus:)

1. LC 103 — Binary Tree Zigzag Level Order Traversal
2. LC 1162 — As Far from Land as Possible
3. LC 934 — Shortest Bridge
4. LC 1926 — Nearest Exit from Entrance in Maze
5. LC 1696 — Jump Game VI
6. LC 310 — Minimum Height Trees
7. GfG — First negative integer in every window of size k
8. LC 733 — Flood Fill
9. GfG — BFS of Graph
10. LC 909 — Snakes and Ladders

Beginner → Advanced Complete Sequence

Follow the Master Roadmap tables above, Level 0 → Level 10, Order 1 → 67. Within a level, problems are already sequenced so each builds on the one before it.

Final Summary

Total curated problems: 67 unique (across LeetCode + GfG; a few are intentional cross-platform reps)

Platform	Count
LeetCode	62
GeeksforGeeks	5

Difficulty	Count (approx.)
Easy	10
Medium	41
Hard	16

Level / Pattern Group	# of Problems
Level 0 — Fundamentals & Implementation	4
Level 1 — Basic Easy (Queue + Arrays/Strings)	4
Level 2 — Queue + Deque	4
Level 3 — BFS on Trees	9
Level 4 — BFS on Graphs	5
Level 5 — Matrix/Grid BFS	3
Level 6 — Multi-Source BFS / Rotting	7
Level 7 — Sliding Window / Monotonic Deque	7
Level 8 — Topological Sort / Kahn's	8
Level 9 — Shortest Path / 0-1 BFS / State-Space	8
Level 10 — Advanced/Hard + Priority Queue + Scheduling	8

Tag	Count (approx.)
 Must-Do	28
 Recommended	21
 Optional	9
 Hard Challenge	9

Final Revision List — Most Important Problems to Re-Solve Before an Interview

One per core pattern, chosen so each forces recall of a genuinely distinct template:

1. **LC 232** — Implement Queue using Stacks (basic mechanics)
2. **LC 622** — Design Circular Queue (wraparound index handling)
3. **LC 102** — Binary Tree Level Order Traversal (the `size = queue.size()` loop, used everywhere after)
4. **LC 200** — Number of Islands (grid-as-graph BFS/DFS)
5. **LC 994** — Rotting Oranges (multi-source BFS — seed the queue with all sources at once)

6. **LC 1091** — Shortest Path in Binary Matrix (single-source shortest path on a grid)
 7. **LC 239** — Sliding Window Maximum (monotonic deque — the two-ended eviction loop)
 8. **LC 207 / 210** — Course Schedule (I & II) (Kahn's Algorithm — in-degree + queue)
 9. **LC 269** — Alien Dictionary (constructing a graph before running Kahn's on it)
 10. **LC 127** — Word Ladder (state-space BFS — nodes that aren't literal grid cells)
 11. **LC 1368** — Minimum Cost to Make at Least One Valid Path in a Grid (0-1 BFS — deque front/back push rule)
 12. **LC 621** — Task Scheduler (queue vs. priority-queue judgment call)
-

Accuracy note: LeetCode numbers/titles were verified against LeetCode's own problem pages and cross-checked against independent, current source listings — including explicit confirmation of premium status for LC 346, 286, 269, and 362/359 (also premium). GfG problem titles were verified against GfG's practice-page and topic-tag listings at the time of writing.